

Labo 03 — Labo pratique : vie, signaux et mort d'un conteneur

Objectif : provoquer vous-même chaque comportement du cours — l'arrêt immédiat, les 10 secondes d'agonie, le code 137, le redémarrage automatique — et voir qui, sans daemon, surveille vos conteneurs.

Prérequis — Labos 01 et 02 terminés. Images `alpine` et `nginx:alpine` présentes.

Fichiers fournis — `files/demarrage-casse.sh` et `files/demarrage-correct.sh`, utilisés à l'étape 3.

Étape 1 — Les états, un par un

```
podman create --name etat alpine sleep 120
podman ps -a --filter name=etat --format 'table {{.Names}}\t{{.Status}}'
```

Observez le statut `Created` : le conteneur existe, aucun processus ne tourne.

```
podman start etat
podman ps --filter name=etat --format '{{.Status}}'
podman pause etat && podman ps -a --filter name=etat --format '{{.Status}}'
podman unpause etat && podman ps --filter name=etat --format '{{.Status}}'
podman stop -t 2 etat && podman ps -a --filter name=etat --format '{{.Status}}'
podman start etat && podman ps --filter name=etat --format '{{.Status}}'
podman rm -f -t 0 etat
```

Observez la succession `Up 1 second`, `Paused`, `Up 5 seconds`, un avertissement `StopSignal SIGTERM failed to stop container etat in 2 seconds, resorting to SIGKILL`, `Exited (137)`, puis de nouveau `Up`.

Explication. Un conteneur `Exited` est redémarrable : il a gardé sa configuration et sa couche d'écriture. Notez que `podman ps` sans `-a` **ne montre pas** un conteneur en pause : il n'est pas « running ».

Étape 2 — Pourquoi un conteneur s'arrête tout seul

```
podman run --name essai1 alpine
podman ps -a --filter name=essai1 --format '{{.Status}}'
```

Observez `Exited (0)` : la commande par défaut d'`alpine` est `/bin/sh`, qui, sans entrée, se termine immédiatement.

```
podman run -d --name essai2 nginx:alpine
podman ps --filter name=essai2 --format '{{.Status}}'
```

Observez `Up` : nginx tourne au premier plan.

```
podman run --rm alpine sh -c 'sleep 60 & echo "lance en arriere-plan"'
```

Observez que la commande revient **immédiatement**, alors qu'un `sleep 60` a bien été lancé.

Explication. Le `&` a détaché `sleep` ; le shell a exécuté `echo` puis s'est terminé. Le PID 1 étant mort, le conteneur est détruit, `sleep` avec lui. C'est le piège numéro un des scripts de démarrage.

Regardez qui surveille `essai2` pendant qu'il tourne :

```
podman inspect --format '{{.State.CommonPid}}' essai2
ps -o pid,ppid,user,comm -p $(podman inspect --format '{{.State.CommonPid}}' essai2)
```

Observez un processus `common`, sous **votre** utilisateur : c'est le superviseur que Podman laisse derrière chaque conteneur — le seul « daemon » qu'il vous reste, et il ne pèse que quelques centaines de Ko.

```
podman rm essai1 ; podman rm -f -t 0 essai2
```

Étape 3 — Le script de démarrage cassé, et sa correction

Copiez les deux scripts fournis :

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <chemin-du-labo>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Exécutez le premier **dans** un conteneur, en montant le dossier courant :

```
podman run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
```

Observez le message affiché, puis un retour immédiat au prompt : le conteneur est déjà mort et supprimé.

```
podman run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
podman ps --filter name=correct --format '{{.Status}}'
podman top correct
```

Observez que le conteneur reste `Up`, et que `podman top` montre `sleep 300` en PID 1 — et **pas** de processus `sh` parent.

Explication. Le `exec` du second script a **remplacé** le shell par la commande finale, qui hérite du PID 1. C'est exactement ce que fait la forme `exec` d'un `ENTRYPOINT`, vue au labo 04.

→ LINUX

`exec` est une commande interne du shell qui appelle l'appel système du même nom : le processus courant abandonne son programme (le shell) et charge le programme demandé **à sa place**, en gardant son PID. Sans `exec`, le shell crée un enfant (`fork`) et attend. Avec `exec`, il n'y a plus de shell du tout.

```
podman rm -f -t 0 correct
```

Étape 4 — Les 10 secondes d'agonie

Mesurez un arrêt sur un processus qui ignore `SIGTERM` :

```
podman run -d --name veille alpine sleep 300
time podman stop veille
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' veille
podman rm veille
```

Observez l'avertissement `StopSignal SIGTERM failed to stop container veille in 10 seconds, resorting to SIGKILL`, `real 0m10.1s` et `code=137 oom=false`.

Recommencez avec un mini-init :

```
podman run -d --init --name veille alpine sleep 300
podman exec veille ps -o pid,comm
time podman stop veille
podman inspect --format 'code={{.State.ExitCode}}' veille
podman rm veille
```

Observez `1 podman-init` puis `sleep` en PID 2, un arrêt en `0m0.1s` et `code=143`.

Et avec une application qui gère correctement ses signaux :

```
podman run -d --name web nginx:alpine
time podman stop web
podman inspect --format 'code={{.State.ExitCode}}' web
podman rm web
```

Observez un arrêt instantané et `code=0`.

Explication. Trois comportements, trois causes. `sleep` en PID 1 **ignore** `SIGTERM` (protection du noyau) : le moteur attend puis tue → `137`. Avec `--init`, `sleep` n'est plus PID 1, l'action par défaut s'applique → `143`. nginx installe un gestionnaire de signal et se termine proprement (avec le code `0`, parce que nginx a choisi de sortir normalement). Ces dix secondes multipliées par vos conteneurs, c'est la durée inexpliquée de vos redéploiements.

Vous pouvez raccourcir le délai de grâce — sans corriger la cause :

```
podman run -d --name veille alpine sleep 300
time podman stop -t 2 veille
podman rm veille
```

Observez `real 0m2.1s`, toujours avec le code `137`.

Étape 5 — Lire les codes de sortie

```
podman run --rm alpine sh -c 'exit 0' ; echo "code=$?"
podman run --rm alpine sh -c 'exit 3' ; echo "code=$?"
podman run --rm alpine commande-absente ; echo "code=$?"
```

Observez `0`, `3`, puis `127` accompagné de `Error: crun: executable file commande-absente not found in $PATH`.

```
podman run -d --name tue alpine sleep 300
podman kill tue
podman inspect --format 'code={{.State.ExitCode}}' tue
podman rm tue
```

Observez `137`, immédiatement cette fois : `kill` n'attend pas.

Provoquez maintenant un vrai manque de mémoire :

```
podman run --name oom --memory=32m --memory-swap=32m alpine sh -c 'head -c 100m /dev/zero |
tail'
echo "code=$?"
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' oom
podman rm oom
```

Observez `code=137` et cette fois `oom=true` : même code, cause différente, et seul `inspect` fait la différence.

Explication. Au-delà de 128, le code indique une mort par signal : `code - 128` donne le numéro du signal. `127` en revanche est une erreur de lancement : l'application n'a jamais démarré.

Étape 6 — `exec` contre `attach`

```
podman run -d --name web nginx:alpine
podman exec web nginx -v
podman exec -it web sh
```

Dans le shell obtenu, tapez :

```
ps -o pid,comm
exit
```

Observez que `nginx` est le PID 1, suivi de ses *workers*, et que votre `sh` porte un autre PID. En sortant du shell, le conteneur est **toujours** `Up`.

```
podman ps --filter name=web --format '{{.Status}}'
```

Explication. `exec` a créé un **nouveau** processus dans les namespaces du conteneur. Le quitter n'affecte pas le PID 1. `attach`, à l'inverse, vous brancherait sur nginx lui-même : un `Ctrl+C` l'arrêterait. Pour consulter les logs, utilisez toujours :

```
podman logs --tail 5 web
podman logs -f --since 1m web      # Ctrl+C ici n'arrête que l'affichage
```

Étape 7 — Les logs ne viennent que de `stdout`

```
podman run --rm --name logs-demo alpine sh -c \
  'echo "je vais sur stdout"; echo "je vais dans un fichier" > /tmp/app.log; sleep 1'
```

Observez que seule la première ligne apparaît.

```
podman run -d --name logs-demo alpine sh -c \
  'echo "visible"; echo "invisible" > /tmp/app.log; sleep 120'
podman logs logs-demo
podman exec logs-demo cat /tmp/app.log
podman rm -f -t 0 logs-demo
```

Observez que `podman logs` affiche `visible` et que le contenu du fichier n'est accessible qu'en entrant dans le conteneur.

Explication. `common` ne capte que `stdout` et `stderr` du PID 1. C'est pourquoi une application conteneurisée doit logger sur la console — et pourquoi il ne faut pas configurer `logging.file.name` dans un Spring Boot conteneurisé.

Étape 8 — Redémarrage automatique

```
podman run -d --restart=on-failure:3 --name instable alpine \
  sh -c 'echo "demarrage $(date +%T)"; sleep 3; exit 1'
sleep 20
podman ps -a --filter name=instable --format '{{.Names}} {{.Status}}'
podman inspect --format 'redemarrages={{.RestartCount}} code={{.State.ExitCode}}' instable
podman logs instable
```

Observez un `RestartCount` de `3`, un statut `Exited (1)`, et **quatre** lignes « demarrage » dans les logs : la tentative initiale plus trois reprises.

Explication. Les logs s'accumulent d'une exécution à l'autre sur le même conteneur : la première ligne est la cause initiale. `.State`, en revanche, ne décrit que la **dernière** exécution.

```
podman rm instable
podman events --since 2m --until 1s | grep instable | awk '{print $5, $6}' | uniq -c
```

Observez le journal des événements : `container start`, `container died`, `container restart` ... C'est le seul endroit où l'on voit l'*histoire* d'un conteneur, pas seulement son état.

Vérifiez enfin l'incompatibilité annoncée en cours :

```
podman run --rm -d --restart=always nginx:alpine
```

Observez `Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"`.

PODMAN

Et après un reboot ? Testez : `podman run -d --restart=always --name survivant nginx:alpine`, puis fermez **toutes** vos fenêtres Ubuntu et, depuis PowerShell, `ws1 --shutdown`. Rouvrez Ubuntu : `podman ps` est vide. Personne n'a relu la politique — il n'y a pas de daemon. Sur un serveur, ce rôle revient à `systemd` via un fichier Quadlet (labo 10). Sur votre poste, c'est un comportement acceptable : vos conteneurs de développement n'ont pas à survivre à un redémarrage. `podman rm -f -t 0 survivant` ensuite.

Étape 9 — Extraire une preuve d'un conteneur mort

```
podman run --name autopsie alpine sh -c 'echo "trace importante" > /rapport.txt; exit 2'
podman ps -a --filter name=autopsie --format '{{.Status}}'
podman cp autopsie:/rapport.txt ./rapport.txt
cat rapport.txt
```

Observez que le fichier est récupérable alors que le conteneur est `Exited (2)`.

```
podman rm autopsie
podman cp autopsie:/rapport.txt ./autre.txt
```

Observez `Error: container "autopsie" does not exist` : le conteneur supprimé, tout est perdu.

Explication. Règle d'exploitation : **on inspecte avant de supprimer**. `cp`, `logs` et `inspect` fonctionnent sur un conteneur arrêté, jamais sur un conteneur supprimé.

Nettoyage

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
podman rm -f -t 0 veille web correct instable autopsie essai1 essai2 tue etat oom survivant
rm -f ~/labo-docker/03/rapport.txt
podman ps -a --format '{{.Names}}'
```

Observez qu'aucun conteneur de ce labo ne subsiste. Les images `alpine` et `nginx:alpine` sont conservées.

Ce que vous devez pouvoir affirmer maintenant

- Un conteneur meurt avec son PID 1 — vous avez provoqué les trois cas.
- `sleep` en PID 1 ignore `SIGTERM` ; `--init` corrige le symptôme, `exec` la cause.
- `137` = tué (par `stop`, `kill` ou l'OOM killer — `inspect` tranche), `143` = arrêté proprement, `127` = commande introuvable.
- `exec` crée un processus, `attach` se branche sur le PID 1.
- `podman logs` ne montre que `stdout` / `stderr`, captés par `common`.

- `podman rm` détruit les logs et les preuves : on inspecte d'abord.
- Sans daemon, une *restart policy* ne survit pas à un `wsl --shutdown`.